# Документация программного обеспечения.

## “Написание ПО приема файлов большого размера”.

***Команда HardBa$$ Росноу г. Москва, РГУ им. А.Н. Косыгина, ОЦ “Инженерия БАС”***

**Состав команды:**

1. Кривова Елизавета Александровна (капитан)
2. Истомин Алексей Витальевич
3. Красина Анжелика Сергеевна
4. Крюков Максим Юрьевич
5. Сухоруких Владислав Игоревич

**Наставник:**

Владимир Алексеевич Зернов

---

# I Описание задачи

**Заказчик:** ООО «Форпост»
**Цель:** Разработать серверное программное обеспечение для приема файлов большого размера (от 1 ГБ и более), работающего на ПК под управлением ОС Linux (Debian 12+).

**Основные требования:**

- Прием и сохранение больших файлов (от 1 ГБ) с возможностью возобновления передачи при разрыве соединения.
- Одновременная работа до 10 клиентов.
- Возможность дозагрузки недостающих частей файла в нескольких сеансах.
- Поддержка нестабильных и медленных каналов связи.
- Возможность указания пути сохранения принимаемых данных на диске.
- Серверное приложение должно быть совместимо с сетевыми протоколами и иметь документацию на взаимодействие с клиентским ПО.

**Ожидаемая себестоимость:** не более 500 000 рублей
**Срок разработки:** 2 месяца
**Рекомендуемая команда:** 6 участников с опытом разработки на Python, Go, C++ для серверных систем на базе UNIX.

**Результаты:**

- Исполняемый файл программы для приема данных.
- Техническая документация на разработанное ПО.

---

# II Инструкция для запуска

## Требования

- Docker
- Docker Compose

## Установка и запуск

1. Клонируйте репозиторий:
   - git clone https://github.com/VladisSuh/HardBassRNUProject.git
   - cd HardBassRNUProject

2. Запустите контейнеры с помощью Docker Compose:

   PORT=9090 STORAGE=./data docker-compose up --build -d

   Переменные окружения:

   - `PORT`: Порт, на котором будет запущен сервер (по умолчанию 6382).
   - `STORAGE`: Путь к директории для хранения данных (по умолчанию `./data`).
3. Остановка контейнеров:
   - docker-compose down
4. Остановка контейнеров и удаление данных:
   - docker-compose down -v
5. Перезапуск контейнеров:
   - docker-compose restart
6. Запуск для клиента:
   - go run cmd/client/main.go -port=6372 -file=example.txt

# III Общая архитектура системы

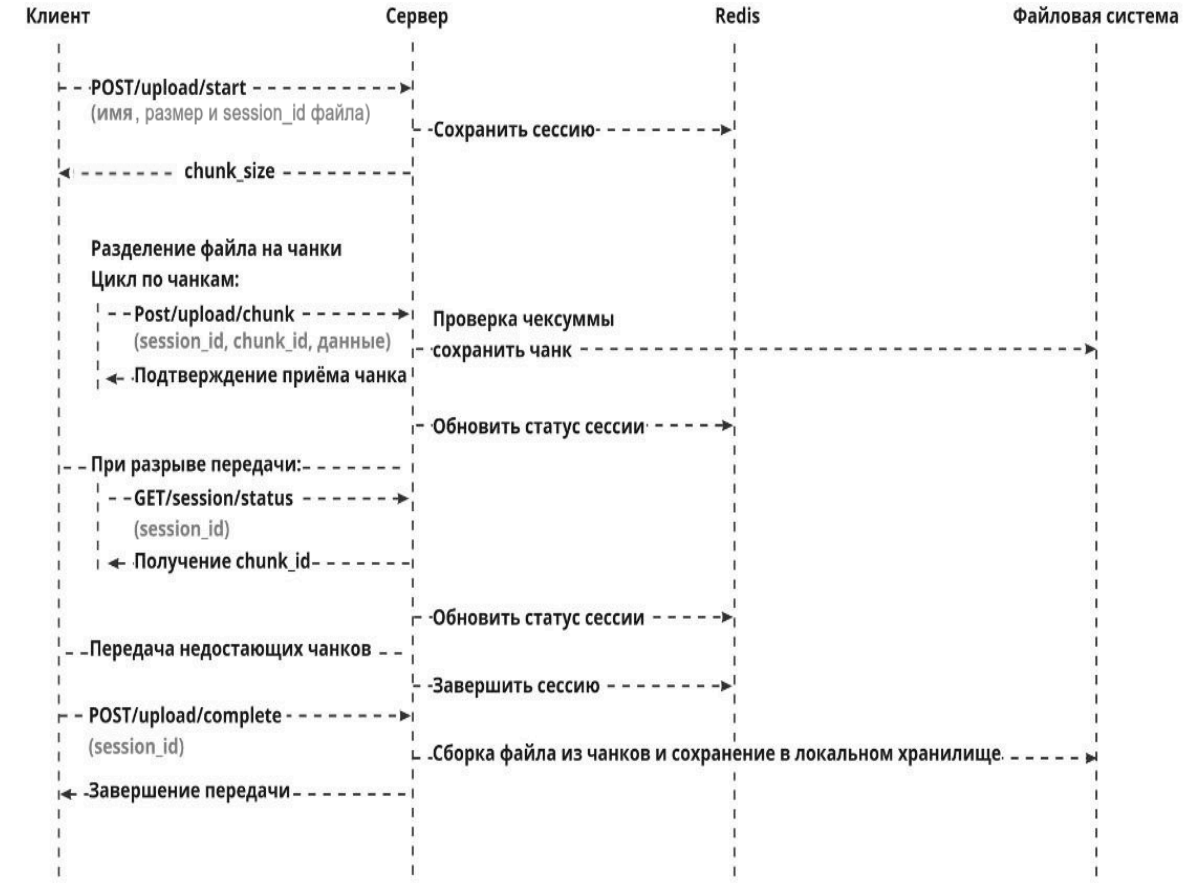
## Основные компоненты

1. **Клиент**: Отправляет файл на сервер, загружая его чанками. Поддерживает механизм дозагрузки в случае разрыва связи.
2. **Сервер (API)**: Принимает файл по частям, проверяет целостность данных с помощью контрольных сумм, сохраняет прогресс загрузки в Redis и обеспечивает восстановление сессии.
3. **Redis**: Хранилище для временных данных о сессиях загрузки, прогрессе и логах. Помогает серверу отслеживать прогресс и восстанавливать загрузку с места остановки.
4. **Файловая система сервера**: Место для сохранения загруженных файлов.



## Основной поток работы:

1. Клиент инициализирует сессию загрузки с сервера. Сервер возвращает размер чанка.
2. Клиент делит файл на чанки и обрабатывает каждый чанк отдельно.
3. Прогресс загружаемых чанков записывается в Redis.
4. Если загрузка прерывается, клиент может запросить информацию о загруженных чанках и продолжить загрузку с места остановки.
5. После полной загрузки файла сервер проверяет целостность, собирает файл и завершает сессию.



# IV Инструменты

- **Go**: Основной язык для разработки сервера и клиента.
- **Redis**: Используется для временного хранения информации о сессиях и прогрессе загрузки.
- **Go HTTP**: Стандартный пакет для обработки HTTP-запросов.
- **Go Redis**: Клиент для работы с Redis.
- **Testify**: Библиотека для написания тестов.
- **Docker**: Для контейнеризации приложения и настройки окружения Redis и сервера.
- **Git**: Система контроля версий для управления кодом.

---

# V Структура API

## 1. POST /upload/start

Начало сессии загрузки файла. Запрашивает у сервера инициализацию загрузки файла.

**Описание:** Этот эндпоинт инициирует новую сессию передачи файла. Генерируется (GUID) для сессии. Клиент получает этот идентификатор для дальнейших операций с чанками.

- Запрос:
  - Метод: `POST`
  - Тело запроса (JSON):

```
{
      "file_name": "example.txt",          // Имя файла
      "file_size": 104857600               // Размер файла в
байтах (например, 100MB)
      "file_hash": "abc123hash"            // Хеш файла

}
```

- Ответ:
  - Код ответа: 200 OK
  - Тело ответа (JSON):

```
{
      "chunk_size": 4194304                                  //
Рекомендуемый размер чанка (например, 4MB)
}
```

  - Действие: сервер создает запись о сессии загрузки в Redis и возвращает параметры для клиента.

- Ошибки:
  - **400 Bad Request** (например, неверные параметры):

```
{
      "status": "error",
      "error_code": 400,
      "message": "Invalid request. Missing or incorrect
parameters.",
      "details": "Parameter 'file_size' is required and must
be a positive integer."
}
```

- **500 Internal Server Error** – Если Redis недоступен для сохранения прогресса или произошла внутренняя ошибка сервера

```
{
      "status": "error",
      "error_code": 500,
      "message": "Internal server error.",
      "details": "Unable to connect to Redis server.",
      "suggestion": "Please try again later."
}
```

---

## 2. POST /upload/{session_id}/chunk

Загрузка одного чанка данных.

**Описание:** chunk_data передается в бинарном формате, поскольку это позволяет избежать проблем с кодировкой и гарантирует, что данные передаются в исходном виде

**Запрос:**

- Метод: `POST`
- URL: `/upload/{session_id}/chunk`
- Тело запроса (формат `multipart/form-data`):
  - Параметры:
    - `chunk_id`: Идентификатор чанка (например, 1).
    - `checksum`: Чек-сумма для проверки целостности чанка.
    - `chunk_data`: Данные чанка в бинарном формате.
  - Пример

```
curl -X POST "http://example.com/upload/{session_id}/chunk" \
  -F "chunk_id=1" \
  -F "checksum=3f786850e387550fdab836ed7e6dc881de23001b" \
  -F "chunk_data=@part1.bin"
```

**Ответ**:

- Успешный:
  - Код: `200 OK`
  - Тело ответа (JSON)

```
{
      "status": "success",
      "message": "Chunk 1 uploaded successfully.",
      "next_chunk_id": 2
}
```

- **Ошибки**
  - **409 Conflict** – Чанк с данным ID уже загружен

```
{
      "status": "error",
      "error_code": 409,
      "message": "Chunk already uploaded.",
      "details": {
      "chunk_id": 1,
      "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
      },
      "suggestion": "Check uploaded chunks via /upload/status
before sending."
}
```

  - **412 Precondition Failed** – Контрольная сумма не совпала

```
{
      "status": "error",
      "error_code": 412,
      "message": "Checksum validation failed.",
      "details": {
      "expected_checksum":
"3f786850e387550fdab836ed7e6dc881de23001b",
      "provided_checksum":
"4e9f0a91e8c9487b5e97e9b9210df7700ac37f77"
      },
      "suggestion": "Please resend the chunk with the correct
data."
}
```

  - **408 Request Timeout** – Таймаут при загрузке чанка

```
{
      "status": "error",
      "error_code": 408,
      "message": "Request timed out. The server did not
receive data in time.",
      "suggestion": "Please retry the request and continue
from the last successful chunk."
}
```

  - **500 Internal Server Error** – Если Redis недоступен для сохранения прогресса или произошла внутренняя ошибка сервера

```
{
      "status": "error",
      "error_code": 500,
      "message": "Internal server error.",
```

```
        "details": "Unable to connect to Redis server.",
        "suggestion": "Please try again later."
    }
```

---

## 3. GET /upload/status/{session_id}

Получение статуса загрузки по сессии.

**Описание:** Возвращает статус загрузки для данной сессии, включая загруженные и ожидающие загрузки чанки.

**Запрос**:

- Метод: `GET`
- URL: `/upload/status/{session_id}`

**Ответ**:

- Успешный:
  - Код: `200 OK`
  - Тело ответа (JSON):

```
{
    "status": "success",
    "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
    "uploaded_chunks": [1, 2, 3],
    "pending_chunks": [4, 5, 6],
    "total_chunks": 10,
    "message": "Upload is in progress."
}
```

**Ошибки:**

- **404 Not Found** – Сессия не найдена:

```
{
    "status": "error",
    "error_code": 404,
    "message": "Upload session not found.",
    "details": {
    "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
    },
    "suggestion": "Ensure that the session ID is correct or
restart the upload."
}
```

---

## 4. POST /upload/complete/{session_id}

Завершение сессии загрузки файла.

**Описание**: Клиент завершает загрузку, уведомляя сервер о том, что все чанки отправлены. Сервер собирает файл, проверяет контрольные суммы и завершает сессию.

**Запрос**:

- Метод: POST
- URL: /upload/complete/{session_id}

**Ответ**:

- Успешный:
  - Код: 200 OK
  - Тело ответа (JSON):

```
{
      "status": "success",
      "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
      "message": "File upload completed successfully."

}
```

**Ошибки:**

- **409 Conflict** – Некоторые чанки не были загружены

```
{
      "status": "error",
      "error_code": 409,
      "message": "File upload incomplete. Some chunks are still
missing.",
      "details": {
      "missing_chunks": [4, 5]
      },
      "suggestion": "Upload the missing chunks before completing the
session."
}
```

---

## 5. GET /upload/logs

Получение логов работы сервера.

**Описание:** Возвращает логи, связанные с загрузками файлов и сессиями.

**Запрос**:

- Метод: `GET`
- URL: `/upload/logs`

**Ответ**:

- Успешный:
  - Код: `200 OK`
  - Тело ответа (JSON):

```
{
      "status": "success",
      "logs": [
      "2024-10-11T12:34:56Z: Session
f47ac10b-58cc-4372-a567-0e02b2c3d479 started.",
      "2024-10-11T12:35:00Z: Chunk 1 uploaded successfully.",
      "2024-10-11T12:36:00Z: Chunk 2 uploaded successfully."
      ]
}
```

---

## 6. DELETE /upload/{session_id}

Удаление сессии загрузки.

**Описание:** Удаляет сессию загрузки и все связанные данные, включая чанки и метаданные.

**Запрос:**

- Метод: `DELETE`
- URL: `/upload/{session_id}`
- Параметры в URL: `session_id` — идентификатор сессии.

**Пример:**

```
      curl -X DELETE
"http://example.com/upload/f47ac10b-58cc-4372-a567-0e02b2c3d479"
```

**Ответ**:

- Успешный:
  - Код: `200 OK`
  - Тело ответа (JSON):

```
{
      "status": "success",
      "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
```

```
        "message": "Upload session deleted successfully."
    }
```

**Ошибки:**

- **404 Not Found** – Сессия не найдена

```
{
    "status": "error",
    "error_code": 404,
    "message": "Upload session not found.",
    "details": {
    "session_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479"
    }
}
```

- **500 Internal Server Error** – Удаление сессии не удалось (например, Redis недоступен)

```
{
    "status": "error",
    "error_code": 500,
    "message": "Failed to delete the upload session. Please try
again later."
}
```

---

# VI Основные модули

# Контроллеры

Контроллеры обрабатывают HTTP-запросы, взаимодействуют с сервисами и возвращают данные клиенту.

Основные функции:

1. Инициализация загрузки(POST upload/start):
    - Обрабатывает запрос, получая хэш файла (SHA-256), имя и размер файла.
    - Передает параметры в сервис создания сессии.
    - Запускает контейнер KeyDB.
    - Возвращает клиенту размер чанка в зависимости от размера файла.
2. Прием чанков (POST upload/{session_id}/chunk):
    - Проверяет существование сессии.
    - Принимает чанки в бинарном виде, проверяя их целостность через чексумму.
    - Сохраняет их в локальном хранилище.
    - Добавляет ID чанка в множество Redis.
    - Обновляет информацию о сессии в Redis.
3. Получение статуса (GET upload/status/{session_id}):
    - Вызывает сервис для получения текущего прогресса загрузки.
4. Завершение загрузки (POST /upload/complete/{session_id})
    - Проверяет, что все чанки успешно загружены.
    - Собирает чанки в единый файл.
    - Удаляет временные файлы после успешной загрузки.
    - Завершает работу клиента.

Файлы:

- start.go — обрабатывает запросы на начальную инициализацию загрузки файла.
- chunk.go — обрабатывает запросы для загрузки чанков.
- status.go — предоставляет статус текущей загрузки.
- complete.go — завершает процесс загрузки.
- delete.go — реализует удаление сессии и связанных данных.

start.go, chunk.go, status.go, complete.go, delete.go — определяют маршруты и функции для обработки вышеописанных операций.

Подробная структура: **internal/handlers**

## start.go

- **StartSession(w http.ResponseWriter, r *http.Request)**

  **Описание:** Запускает новую сессию для загрузки файла, принимая имя файла и его размер.

  **Процесс работы:**

- Извлекает информацию о файле из запроса, включая имя файла и его размер.
- Создаёт сессию с помощью SessionService, связывая её с клиентом.
- Возвращает клиенту уникальный идентификатор сессии, который будет использоваться для передачи чанков данных.

---

## chunk.go

- **UploadChunk(w http.ResponseWriter, r *http.Request)**

  **Описание:** Загружает один чанк (фрагмент файла) в рамках существующей сессии. Эта функция обрабатывает загрузку части файла, проверяя идентификатор сессии и валидность полученных данных. В случае успеха добавляет фрагмент в хранилище, иначе возвращает сообщение об ошибке.

  **Процесс работы:**

  - Проверяет идентификатор сессии.
  - Извлекает данные чанка из запроса и передаёт их в сервис для сохранения.
  - Проверяет наличие ошибок при сохранении и отправляет соответствующий HTTP-ответ клиенту.

---

## status.go

- **GetUploadStatus(w http.ResponseWriter, r *http.Request)**

  **Описание:** Получает текущий статус загрузки файла по идентификатору сессии. Функция обращается к сервису сессий для получения состояния загрузки и отправляет JSON-ответ с подробной информацией о прогрессе загрузки.

  **Процесс работы:**

  - Извлекает sessionID из параметров запроса.
  - Вызывает метод GetUploadStatus из SessionService для получения текущего состояния загрузки.
  - Форматирует ответ в виде JSON с детальной информацией о статусе и прогрессе загрузки.
  - Отправляет сформированный ответ обратно клиенту с соответствующим статусом HTTP.

---

## complete.go

- **CompleteUpload(w http.ResponseWriter, r *http.Request)**

**Описание:** Создаёт новый экземпляр UploadChunkHandler для обработки загрузки чанков файлов, используя указанный сервис сессий.

**Процесс работы:**

- Извлекает идентификатор сессии из запроса.
- Проверяет состояние каждого загруженного чанка для этой сессии.
- Если все чанки успешно загружены, инициирует процесс сборки файла на стороне сервера.
- Отправляет клиенту ответ о завершении сессии или сообщает о возникшей ошибке, если загрузка неполная или произошла ошибка при сборке.

---

### delete.go

- **DeleteSession(w http.ResponseWriter, r *http.Request)**

  **Описание:** Удаляет существующую сессию загрузки по её sessionID. Если сессия не найдена, возвращает ошибку 404, иначе завершает удаление сессии и возвращает сообщение об успешном удалении.

  **Процесс работы:**

  - Извлекает sessionID из URL, используя mux.Vars для доступа к параметрам.
  - Проверяет, существует ли sessionID. Если отсутствует, отправляет ответ с ошибкой 400 и сообщением о пропущенном sessionID.
  - Вызывает метод DeleteSession из SessionService, чтобы попытаться удалить сессию.
  - Если сессия не найдена, отправляет ответ с ошибкой 404.
  - Если возникла другая ошибка, возвращает ошибку 500 с сообщением о неудачной операции.
  - Если удаление прошло успешно, отправляет ответ со статусом 200 и JSON-подтверждением успешного завершения.

---

## Сервисы

Сервисы реализуют основную бизнес-логику, включая управление процессом загрузки, проверку данных и взаимодействие с хранилищем.

Основные функции:

1. Создание сессии:
   - Создание записи о сессии в Redis с указанием состояния загрузки (инициализация, ожидание чанков и т.д.).

    - Вычисление и предоставление размера чанков в зависимости от размера файла.
2. Проверка чанков:
    - Проверка корректности данных через чексумму (SHA-256) после загрузки каждого чанка.
    - Верификация размеров и последовательности чанков.
    - Сохранение информации о загруженных чанках в хранилище.
3. Обновление статуса загрузки:
    - Отслеживание прогресса загрузки на основе информации о принятых чанках.
    - Подготовка данных для возвращения клиенту (например, процент завершения загрузки).
4. Завершение загрузки:
    - Проверка, что все ожидаемые чанки были успешно загружены.
    - Сборка всех загруженных чанков в единый файл.
    - Обработка ошибок при сборке, удаление временных данных.
    - Обновление статуса сессии до завершенного состояния.

Файлы:

- file.go — работа с файлами и валидация данных.
- session.go — управление сессиями загрузки.

Подробная структура: **internal/services**

## file.go

- **AssembleChunks(sessionID string, outputFilePath string) -> error**

  **Описание:** Собирает все чанки, принадлежащие одной сессии, в один файл.

  **Процесс работы:**

  - Открывает файл для записи результата.
  - Получает все файлы чанков для сессии.
  - Сортируем файлы по chunkID.
  - Собираем все чанки в один файл.

  **Пример:**

  ```
  err := fileService.AssembleChunks(sessionID,
  "/path/to/output/file")
  ```

---

- **SaveChunk(sessionID string, chunkID int, chunkData []byte) -> error**

  **Описание:** Сохраняет чанк файла как в Redis, так и на диске, обновляя прогресс загрузки.

**Процесс работы:**

- Проверяет существование чанка в Redis.
- Сохраняет данные чанка в Redis и записывает на диск.
- Обновляет общий прогресс загрузки.

---

### session.go

- **CreateSession(fileName string, fileSize int64) -> (string, int64, error)**

  **Описание:** Создаёт новую сессию для загрузки файла, определяет оптимальный размер чанка и сохраняет данные сессии в хранилище.

  **Процесс работы:**

  - Проверяет, что имя файла не пустое и размер файла больше нуля.
  - Генерирует новый идентификатор сессии и рассчитывает размер чанка, основываясь на максимальном размере в 1 GB.
  - Формирует данные сессии, включая имя файла, размер файла, рассчитанный размер чанка, текущий прогресс загрузки (нулевой при создании сессии) и статус сессии ("in_progress").
  - Сохраняет данные сессии в хранилище через Storage.SaveSession.

---

## Хранилище

Управляет взаимодействием с базой данных Redis, которая используется для хранения информации о сессиях загрузки, прогрессе и временных данных.

Основные функции:

1. Инициализация и подключение:
   - Устанавливает подключение к Redis-серверу.
   - Настраивает параметры соединения (адрес, порт, таймауты).
2. Управление сессиями:
   - Создание новой записи в Redis для сессии загрузки.
   - Обновление информации о состоянии сессии (например, добавление ID загруженных чанков).
   - Проверка текущего состояния сессии, включая завершенность.
3. Работа с данными чанков:
   - Сохранение ID и статуса загруженных чанков.
   - Получение списка загруженных чанков для конкретной сессии.
   - Проверка наличия всех необходимых чанков для завершения загрузки.
4. Очистка временных данных:
   - Удаление данных сессии и чанков после успешного завершения загрузки.

- Освобождение ресурсов Redis.

Реализация:

- Модуль работает через стандартные Redis-команды (SET, GET, SADD, SMEMBERS и т.д.) для управления данными.

Файлы:

- redis.go — обеспечивает подключение и операции с Redis.

Подробная структура: **internal/storage**

## redis.go

- **NewRedisClient(addr, password string, db int) -> *RedisClient**

  **Описание:** Инициализирует и возвращает новый клиент Redis.

  **Процесс работы:**

  - Создаёт клиента Redis с заданными параметрами подключения..

---

- **SaveSession(sessionID string, sessionData map[string]interface{}) -> error**

  **Описание:** Сохраняет данные сессии в Redis.

  **Процесс работы:**

  - Использует команду HMSet для записи данных сессии в Redis.

---

- **SaveChunkData(sessionID string, chunkID int, chunkData []byte) -> error**

  **Описание:** Сохраняет данные чанка в Redis и добавляет идентификатор чанка в множество загруженных чанков.

  **Процесс работы:**

  - Сохраняет данные чанка с уникальным ключом и добавляет chunkID в множество загруженных чанков.

---

# Утилиты

Утилиты предоставляют вспомогательные функции для работы с файлами, данными и идентификаторами. Эти функции поддерживают основные процессы системы, такие как валидация данных и генерация уникальных значений.

Основные функции:

1. Проверка целостности данных:
    - Реализует функции вычисления хеша (SHA-256) для входящих данных.
    - Сравнивает вычисленный хеш с переданным клиентом для валидации целостности чанков.
2. Работа с чексуммами:
    - Рассчитывает хеш файла или чанка.
    - Проверяет корректность данных перед их сохранением.

3. Упрощение работы с домашними директориями в Linux дистрибутивах:
    - Расширяет путь, начинающийся с ~, до полного пути к домашнему каталогу.

Реализация:

- Модуль использует стандартные библиотеки для работы с хешами (например, crypto/sha256) и генерации UUID.

Файлы:

- checksum.go — функции для вычисления и проверки хешей.
- path.go — функция для расширения пути для домашнего каталога

Подробная структура: **internal/utils**

## checksum.go

- **ValidateChecksum(data []byte, expectedChecksum string) -> bool**

  **Описание:** Создаёт и инициализирует новую службу проверки контрольной суммы.

  **Процесс работы:**

  - Вычисляет контрольную сумму данных методом CalculateChecksum.
  - Сравнивает полученную и ожидаемую контрольные суммы.

---

- **CalculateChecksum(data []byte) -> string**

**Описание:** Вычисляет контрольную сумму для данных с использованием алгоритма SHA-256.

**Процесс работы:**

- Вычисляет SHA-256 хэш данных и преобразует его в строку шестнадцатеричного формата.

---

### path.go

- **ExpandPath(path string) -> (string, error)**

  **Описание:** расширяет путь, начинающийся с ~, до полного пути к домашнему каталогу.

  **Процесс работы:**

  - Функция проверяет, начинается ли путь с символа ~. Если да, она получает домашний каталог с помощью и затем объединяет домашний каталог с остальной частью пути

---

## Конфигурация

Отвечает за настройку и управление параметрами приложения. Он обеспечивает централизованное хранение настроек, их загрузку и доступ к ним для других компонентов системы.

Основные функции:

1. Загрузка конфигурации:
   - Чтение параметров из файла config.yaml при запуске приложения, если не заданы аргументы командной строки.
   - Поддержка обновления конфигурации (при необходимости).
2. Обработка аргументов командной строки:
   - Возможность переопределения параметров с помощью аргументов командной строки.
   - Учет специфичных настроек для запуска сервера
3. Предоставление настроек:
   - Доступ к параметрам конфигурации из любого модуля приложения через структуры данных.
   - Валидация значений (например, проверка корректности портов или путей).

Реализация:

- Модуль использует библиотеки для парсинга yaml конфигурации

Файлы:

- config.yaml — файл с конфигурацией приложения.
- config.go — реализует логику загрузки и хранения конфигурации.

Подробная структура: **config**

## config.go

- **LoadConfig(path string) -> (*Config, error)**

  **Описание:** Загружает конфигурацию сервера из указанного YAML файла и возвращает заполненную структуру Config.

  **Процесс работы:**

  - Открытие файла.
  - Парсинг данных YAML.
  - Проверка данных конфигурации.
  - Отправляет HTTP POST запрос на сервер.
  - Проверяет ответ сервера для подтверждения успешного завершения загрузки.

  **Пример структуры конфигурации в формате YAML:**

```
server:
  port: 6382

redis:
  addr: "keydb:6379"
  password: "password"
  db: 0

storage:
  path: "./data"
```